Case Study: One Shared REST API for Web and Mobile Clients

Unit 3 · Activity 3 — Web Services and Microservices in Action
Selected case: Case 4, plus application to the DevPath project

Item	Detail
Course	Desarrollo Web Integral (IDGSBIS92N)
Period	SD2026
Activity	Unit 3 · Activity 3
Strategy	Case study
Modality	Team project
Due date	Week 9
Selected case	Case 4: "A mobile app and a web app duplicate the same business rules and drift apart."
Proposed solution	One REST API consumed by both clients
Project	DevPath
Team	Omar Rodriguez, Betsy Durán, Emanuel de la Cerda, Zaid Legarda, Karen Terrazas.



Table of contents

- Part A — Characteristics of web services
  - A1. Web services and APIs
  - A2. Protocols, styles, and data formats
  - A3. Contracts: WSDL and OpenAPI
  - A4. Security of web services
  - A5. Monolith vs. microservices
- Part B — Case analysis (Case 4)
  - B1. Actors
  - B2. Current problem
  - B3. Proposed services
  - B4. Architecture diagram
  - B5. Data exchanged
  - B6. Failure modes
  - B7. Security
  - B8. Trade-offs
- Part C — Application to our project (DevPath)
- References

Part A — Characteristics of web services

A1. Definition of web service and API

Web service

A **web service** is a software system that offers data or functions to other programs over a network, using web technologies. A program sends a request, the service processes it, and the service sends back a response. No person needs to read a web page in the middle of this exchange.

The term has a narrow and a broad meaning. In the original sense, "web services" meant services described with WSDL and exchanged with SOAP messages (Chinnici et al., 2007). Today the term is also used for any service reachable over HTTP, including REST APIs. In this document we use the broad meaning.

API

An **Application Programming Interface (API)** is a defined way for one piece of software to ask another piece of software to do something. It describes which operations exist, what input each operation needs, what output it returns, and which errors can happen. APIs exist at many levels: the functions of a code library, the system calls of an operating system, or the operations of a remote service. A **web API** is an API that is used through HTTP.

Relationship and differences

A web service is the running system. Its API is the interface that other programs use to reach it. A useful comparison is a restaurant: the kitchen is the service, and the menu is the API. Customers do not enter the kitchen; they use the menu to order.

Aspect	Web service	API
What it is	A running system that provides functionality over a network	The set of rules and operations used to access functionality
Network	Always uses a network	May be local (library) or remote (web API)
Relationship	Exposes an API	May belong to a web service, a library, or an operating system
Example	A university enrollment system running on a server	`GET /api/v1/courses` and its JSON response

In short, every web service exposes a web API, but not every API belongs to a web service.

How applications communicate

Web services communicate through HTTP. In HTTP, a client builds a request message that states what it wants, sends it to a server, and then checks the response to see whether its intention was carried out (Fielding et al., 2022). A request contains a method, a URL, headers, and sometimes a body. A response contains a status code, headers, and sometimes a body.

```
GET /api/v1/courses?semester=2026B HTTP/1.1
Host: api.university.example.edu
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "items": [ { "id": "c101", "name": "Web Development" } ] }
```

Practical examples

Example	Who calls whom	What is exchanged
Weather API	A mobile app calls a weather service	City name in, temperature as JSON out
Payment gateway	An online store's server calls a payment provider	Order reference in, payment status out
University API	A student portal calls the registrar's service	Student ID in, course list out
Case 4 (this document)	A web app and a mobile app call the same business API	Requests with JSON data in, results with JSON data out

---

A2. Protocols and styles: SOAP, REST, GraphQL. Formats: XML, JSON

These technologies are often listed together, but they are not the same kind of thing. SOAP is a **protocol**, REST is an **architectural style**, GraphQL is a **query language with a runtime**, and XML and JSON are **data formats**.

SOAP

SOAP (originally "Simple Object Access Protocol", but now just a name) is a protocol for exchanging structured information between systems. It defines an extensible messaging framework, uses XML technologies, and can run over several underlying protocols (Gudgin et al., 2007). A SOAP message has an **Envelope** that contains an optional **Header** and a required **Body**. Errors are reported with a **Fault** element inside the Body. SOAP services are normally described with a WSDL file. Extensions such as WS-Security add features like message-level signing and encryption.

REST

REST (Representational State Transfer) is an architectural style described by Roy Fielding (2000). It is not a protocol or a standard. A REST API organizes the system as **resources**, each identified by a URL. Clients work with **representations** of those resources, usually JSON, and use the standard HTTP methods. Main REST constraints include client–server separation, stateless requests, cacheable responses, a uniform interface, and a layered system (Fielding, 2000).

HTTP gives the methods their meaning (Fielding et al., 2022). For example, `GET` reads, and `POST` creates or processes. `GET`, `PUT`, and `DELETE` are idempotent (repeating them has the same intended effect as sending them once), while `POST` is not guaranteed to be idempotent (MDN Contributors, n.d.-b).

A good REST URL names a thing, not an action: `GET /bookings` is correct, while `/getBookings` is not.

GraphQL

GraphQL is a query language for APIs and a server-side runtime that executes queries against a type system defined by the service (GraphQL Foundation, n.d.). The client describes exactly which fields it needs, and the server returns exactly that shape. A GraphQL API usually exposes a single HTTP endpoint, and most requests are sent with `POST`.

XML and JSON

XML (Extensible Markup Language) is a markup language for structured documents, defined by the W3C (Bray et al., 2008). **JSON** (JavaScript Object Notation) is a lightweight, text-based format for data exchange (Bray, 2017).

> **Important:** XML and JSON are *data formats*. They describe how data is written. They do not define how messages are sent, which operations exist, or how errors are handled. SOAP uses XML, while REST APIs most often use JSON, but a REST API could also return XML.

Comparison of SOAP, REST, and GraphQL

Characteristic	SOAP	REST	GraphQL
Type	Protocol	Architectural style	Query language and runtime
Typical format	XML	JSON (also XML, others)	JSON responses
Transport	HTTP, SMTP, and others	HTTP	Usually HTTP
Endpoints	Usually one service endpoint, many operations	Many URLs, one per resource	Usually one endpoint

Characteristic	SOAP	REST	GraphQL
Contract	WSDL	OpenAPI (optional but recommended)	Schema (type system)
Main advantages	Formal contract, standard error model, WS-Security extensions	Simple, uses HTTP features such as caching, widely supported by web and mobile tools	Client chooses fields, avoids over-fetching and under-fetching, strongly typed
Main disadvantages	Verbose XML, heavier tooling, steeper learning curve	Fixed response shapes can cause over-fetching or several round trips	HTTP caching is harder, complex queries need depth/cost limits, more server complexity
Best use cases	Enterprise or legacy integration that needs formal contracts and message-level security	Web and mobile backends, public APIs, most student and business projects	Complex client screens that need flexible, nested data from many sources

Comparison of XML and JSON

Characteristic	XML	JSON
Syntax	Tags and attributes	Objects, arrays, strings, numbers, booleans, null
Size and readability	More verbose	Shorter and easy to read
Schema language	XML Schema (XSD)	JSON Schema
Native support in JavaScript	Needs a parser	Built in (`JSON.parse`)
Strengths	Namespaces, mixed content, mature document tooling	Simple, compact, natural for web and mobile apps
Typical use	SOAP, document formats, legacy systems	REST APIs, configuration, web and mobile apps

A3. Contracts: WSDL and OpenAPI

What is a service contract?

A **service contract** is a formal, machine-readable description of what a service offers and how to use it. It states which operations exist, what each one accepts, what each one returns, and which errors it can produce. Both the team that builds the service and the teams that use it agree on this document.

WSDL

The Web Services Description Language (WSDL) is an XML language for describing web services (Chinnici et al., 2007). A WSDL document describes the data types, the operations and their messages, the protocol binding (for example, SOAP over HTTP), and the network address of the service. Tools can read a WSDL file and generate client code automatically.

OpenAPI

The OpenAPI Specification (OAS) is a standard way to describe HTTP APIs, including REST APIs. Its goal is that people and tools can understand how to call a service without trial and error (OpenAPI Initiative, 2021). An OpenAPI document is written in YAML or JSON, and the specification recommends naming the root file openapi.json or openapi.yaml. Version 3.1 uses JSON Schema to describe data (OpenAPI Initiative, 2021).

Comparison

Aspect	WSDL	OpenAPI
Description language	XML	YAML or JSON
Designed for	SOAP and XML web services	HTTP and REST APIs
Describes	Types, operations, bindings, endpoints	Paths, methods, parameters, schemas, responses, security
Data typing	XML Schema	JSON Schema
Readability	Verbose, usually read through tools	Compact and readable by people
Typical tools	Code generators for SOAP clients	Documentation viewers, mock servers, validators, code generators

Why contracts matter when teams collaborate

- **Parallel work.** Frontend and backend teams can start at the same time, because both follow the same agreed interface.
- **Fewer misunderstandings.** Field names, types, and error formats are written down instead of assumed.
- **Automation.** Documentation, mock servers, client code, and tests can be generated or validated from the contract.
- **Safe change.** A change to the contract is visible in a pull request, so breaking changes can be discussed before they reach users.

This is directly relevant to Case 4: when two clients depend on one API, the contract is the shared agreement that keeps both clients working.

Elements of a contract

A REST contract usually describes the following:

- **Endpoints:** the URL path and the HTTP method, such as `GET /api/v1/workshops`.
- **Parameters:** path parameters (/workshops/{id}), query parameters (`?available=true`), headers, and request bodies.
- **Schemas:** the structure and types of the JSON that is sent and received.
- **Responses:** what the API returns for each outcome.

- **HTTP status codes:** the result category of the request (Fielding et al., 2022).

A short OpenAPI example:

```yaml
openapi: 3.1.0
info:
 title: Campus Workshops API
version: 1.0.0
paths:
 /api/v1/workshops:
      get:
        summary: List workshops
        parameters:
          - name: available
            in: query
            schema:
              type: boolean
         responses:
           "200":
             description: A list of workshops
             content:
               application/json:
                 schema:
                   type: object
                   properties:
                     items:
                       type: array
                       items:
                         $ref: "#/components/schemas/Workshop"
           "401":
             description: Missing or invalid credentials
components:
schemas:
      Workshop:
        type: object
        required: [id, title, seatsAvailable]
        properties:
          id: { type: string }
          title: { type: string }
          seatsAvailable: { type: integer, minimum: 0 }
```

Common HTTP status codes

Code	Meaning	Typical use
200 OK	The request succeeded	Successful `GET`, or a successful operation that returns data
201 Created	A new resource was created	Successful `POST` that creates something
204 No Content	Success with no response body	Successful `DELETE`
400 Bad Request	The request is malformed or invalid	Broken JSON, missing required field
401 Unauthorized	Valid authentication credentials are missing	No token, expired token

Code	Meaning	Typical use
403 Forbidden	The server understood the request but refuses it	User is authenticated but not allowed
404 Not Found	The resource does not exist	Unknown ID
409 Conflict	The request conflicts with the current state	Duplicate registration, no seats left
422 Unprocessable Content	The format is correct but the content is not acceptable	Field value breaks a business rule
429 Too Many Requests	The client sent too many requests	Rate limit exceeded
500 Internal Server Error	Unexpected server failure	Unhandled exception
503 Service Unavailable	The server cannot handle the request now	Maintenance or overload

Sources: Fielding et al. (2022) and Nottingham and Fielding (2012). Both 400 and 422 can describe invalid input. Teams should choose one convention and document it in the contract.

A4. Security of web services

Before looking at each mechanism, it is important to separate two ideas:

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
Example	Checking a password or a token	Checking that a student can view only their own registrations
Typical failure response	401 Unauthorized	403 Forbidden

A request can be authenticated but still not authorized. Both checks are needed.

HTTPS

- **Purpose:** protect data while it travels between client and server. HTTPS is HTTP carried over TLS, and it gives confidentiality, integrity, and server authentication (Fielding et al., 2022; Rescorla, 2018).
- **How it works:** the client and the server perform a TLS handshake, the server presents a certificate, and both sides create keys to encrypt the traffic. The `https` URL scheme tells the client to use TLS.
- **Benefits:** prevents eavesdropping and tampering on the network, and lets the client check that it is talking to the real server.
- **Limitations:** it protects data only *in transit*. It does not protect stored data, does not identify the user, and does not stop attacks that use valid requests. Certificates must be issued and renewed correctly.

API keys

- **Purpose:** identify the *application* (not the person) that is calling an API.
- **How it works:** the server issues a secret string to each client application. The client sends it with each request, usually in a header such as `X-API-Key`. The server looks it up and accepts or rejects the call.
- **Benefits:** very simple to implement, useful for service-to-service calls, usage tracking, and basic rate limiting per client.
- **Limitations:** a key is a static secret. If it leaks (for example, inside mobile app code or a public repository), anyone can use it. It does not identify the end user and has no built-in permission model or expiry. Keys must be sent only over HTTPS, stored in environment variables or a secrets manager, and rotated.

OAuth 2.0

- **Purpose:** an **authorization framework** that lets an application obtain *limited access* to a protected service, without receiving the user's password (Hardt, 2012).
- **How it works:** there are four roles: the resource owner (user), the client (application), the authorization server, and the resource server (API). The client obtains an **access token** from the authorization server, then sends it to the API. Tokens can have **scopes** that limit what they allow. For public clients such as mobile apps and single-page web apps, the Authorization Code flow is used with PKCE (Sakimura et al., 2015).
- **Benefits:** the user's password is not shared with every application, access can be limited by scope and time, and access can be revoked.
- **Limitations:** it is complex, and mistakes in configuration are common. OAuth 2.0 itself handles *authorization*, not user identity. Identity is added by layers such as OpenID Connect.

JWT

- **Purpose:** a compact, URL-safe **token format** for representing claims between two parties (Jones et al., 2015).
- **How it works:** a signed JWT has three Base64url-encoded parts separated by dots: header, payload (claims such as `sub` for the subject and `exp` for expiry), and signature. The server that receives the token verifies the signature with a secret or public key. If the signature is valid and the token has not expired, the server can trust the claims without a database lookup.
- **Benefits:** stateless verification, works well across several services, and carries useful claims such as user ID and role.
- **Limitations:** the payload is encoded, **not encrypted**, so anyone who has the token can read it. Sensitive data must not be put inside. A token cannot be easily revoked before it expires, so access tokens should be short-lived. Implementations must verify the signature, the algorithm, and the expiry, as recommended in the JWT best current practices (Sheffer et al., 2020).

> **JWT and OAuth 2.0 are not the same thing.** OAuth 2.0 is a framework that describes how tokens are requested and used. JWT is one possible format for those tokens. An OAuth 2.0 server can issue JWT access tokens, but it can also issue opaque random strings. A system can also use JWT without OAuth 2.0.

Rate limiting

- **Purpose:** limit how many requests a client can send in a period of time, to protect the API from overload, abuse, and brute-force attacks.
- **How it works:** the server counts requests per client (by API key, user, or IP address). Common algorithms are fixed window, sliding window, and token bucket. When the limit is exceeded, the server answers `429 Too Many Requests`, and it may include a `Retry-After` header that says how long the client should wait (MDN Contributors, n.d.-a; Nottingham & Fielding, 2012).
- **Benefits:** keeps the service available, reduces the effect of bots and runaway clients, and controls cost.
- **Limitations:** IP-based limits can block many legitimate users who share one address, such as a campus network. Distributed attackers can stay under per-IP limits. Counting across several server instances needs shared storage. Rate limiting does not replace authentication.

Summary

Mechanism	Main job	Answers the question	Key weakness
HTTPS	Protect data in transit	Is the channel private and is this the right server?	Does not protect stored data or identify users
API key	Identify an application	Which app is calling?	Static secret that can leak
OAuth 2.0	Delegate limited access	What may this app do on behalf of the user?	Complex to configure correctly
JWT	Carry signed claims	What does this token say, and is it valid?	Readable payload, hard to revoke early
Rate limiting	Control request volume	Is this client sending too much?	Cannot tell good from bad traffic by itself

---

A5. Monolith vs. microservices

A **monolith** is an application built and deployed as one unit. Its modules run in the same process and usually share one database. **Microservices** are an architecture style in which an application is built as a collection of small, autonomous services. Each service focuses on a single business capability and can be developed and deployed independently (Microsoft, n.d.-b).

Comparison

Dimension	Monolith	Microservices
Architecture and organization	One codebase, one deployable unit, internal modules or layers	Several small services, each with its own codebase and, ideally, its own data
Advantages	Simple to build, test, and run; easy debugging; one place for transactions; low infrastructure cost	Independent deployment and scaling; teams can work separately; one service can use a different technology; a failure can stay local
Disadvantages	The whole application must be deployed together; a large codebase can become hard to understand; one technology for everything	Much higher complexity; distributed data and consistency problems; network latency; harder testing; more infrastructure
Scalability	Scale the whole application, even if only one part is busy	Scale only the service that needs it
Maintenance	Easy at small size; becomes difficult if modules are tightly coupled	Each service is small, but there are many services, versions, and contracts to maintain
Deployment	One pipeline, one release	Many pipelines; needs automation (CI/CD) and often containers
Communication	Function calls inside the same process (fast and reliable)	Network calls (HTTP, messaging); slower and can fail (Microsoft, n.d.-b)
Monitoring	One set of logs and metrics	Needs centralized logging, metrics, and distributed tracing to follow a request across services
Fault tolerance	A serious bug or crash can stop the whole application	Failures can be isolated, but only if the design handles timeouts, retries, and fallbacks
Appropriate use cases	Small and medium systems, small teams, early-stage products, clear and stable scope	Large systems, many teams, parts with very different scaling needs, strong need for independent releases

A useful middle option is the **modular monolith**: one deployable application with clear internal modules and layers, which can be split later if the need appears.

Why a shared REST API solves Case 4 without multiple microservices

The problem in Case 4 is **duplicated business rules**, not a lack of scalability or team independence. The fix is to have *one* place that owns those rules. A shared REST API does this by exposing the rules through HTTP so that every client uses the same logic.

Because of this, the solution only needs:

1. **One backend** with a REST API,
2. **A business logic layer** inside that backend,
3. **A data access layer** and one database.

It does not need separate "validation service", "pricing service", and "registration service" processes. Splitting the backend into many services would add network calls, distributed data, and monitoring work without fixing the original problem any better. A single, well-layered backend is the proportionate solution, and it can still be split later if the system grows to need it.

Part B — Case analysis (Case 4)

Problem: "A mobile app and a web app duplicate the same business rules and drift apart." **Proposed solution:** One REST API consumed by both clients.

To make the analysis concrete, this part uses an **illustrative example**: a *Campus Workshop Registration* system. Students can browse workshops, register, and pay a fee. The system has a web application and a mobile application. This example is only a teaching scenario for Case 4. It is **not** our DevPath project, which is covered in Part C.

Business rules in this example:

- A workshop has a limited number of seats.
- Registration closes at a deadline. After a "late registration" date, a **10% late fee** is added to the price.
- A student cannot register twice for the same workshop.
- Phone numbers must follow one accepted format.

B1. Actors

Actor	Type	Role in the system
Web application users	Person	Students and staff who use the system from a browser. They browse workshops, register, and view their registrations.
Mobile application users	Person	The same kind of people, using the mobile app. They need the same results as web users.
Web application	Client software	Shows the user interface in the browser. Collects input, sends requests to the API, and shows the results.
Mobile application	Client software	Does the same on a phone. It also handles mobile concerns such as secure token storage and unstable connections.
Shared REST API	Server interface	The single entry point for both clients. Receives HTTP requests, authenticates the caller, validates the request format, calls the business logic, and returns JSON with an HTTP status code.
Backend business logic	Server software	The authoritative implementation of the rules: validations, calculations (such as the late fee), permissions, and workflows. Lives only on the server.
Database	Data store	Stores persistent data: users, workshops, registrations, and fees. Only the backend can access it.

Additional component, included because it is technically justified:

Component	Why it is justified
Reverse proxy or load balancer (infrastructure)	Terminates HTTPS, can apply rate limits, and lets several backend instances share traffic. It is part of deployment, not part of the business design, so it is not drawn in the main diagram.

No further services, queues, or caches are added, because the case does not require them.

B2. Current problem

Today, the web app and the mobile app each contain their own copy of the business rules. At the beginning the copies are the same. Over time they drift apart, because different developers change them at different times, in different languages, and with different tests.

Duplicate business logic

Each client validates input, calculates totals, and decides what is allowed. Every rule exists twice, and every change must be made twice.

Realistic examples of inconsistency

Rule	Web app behavior	Mobile app behavior	Result
Phone validation	Accepts exactly 10 digits	Accepts an optional `+52` prefix and spaces	The same phone number is accepted on one device and rejected on the other
Late fee (10% of price)	Rounds the fee to whole pesos: 10% of 135.00 becomes 14	Keeps two decimals: 10% of 135.00 is 13.50	The student pays 149.00 on the web and 148.50 on mobile
Registration deadline	Uses the browser's local time	Uses UTC	A student registering at 23:30 local time is on time on the web but late on mobile
Seat availability	Uses the seat count loaded when the page opened	Checks seats again before sending	The web app can show a seat that no longer exists

Maintenance difficulties

- A rule change needs two implementations, two sets of tests, and two releases.
- Web changes are published immediately, but a mobile update must pass app-store review, and users update at different times. For some period, **several versions of the rule are active at once**.
- Bug fixes can be applied in one client and forgotten in the other.
- Developers must understand two code bases to answer a simple question like "How is the late fee calculated?"

Impact on users

- Confusing and unfair results depending on the device.
- Loss of trust, support requests, and complaints about "wrong" fees.

- Registrations that succeed on one platform but fail on the other for the same data.

Impact on developers

- Duplicate work and duplicate bugs.
- Long investigations to find out which client is "correct".
- Fear of changing rules, because the effects on the other client are hard to predict.

Risks to data consistency

If each client enforces the rules by itself, the **database has no single guardian**. Possible effects include:

- **Overbooking:** two clients read "1 seat left" at the same time, and both create a registration.
- **Duplicate registrations** for the same student and workshop.
- **Different totals** stored for the same product, which breaks financial reports.
- **Invalid data** reaching the database, because one client skipped a check that the other one performs.

Rules that run only in a client cannot be trusted, because clients can be modified, can be outdated, or can be bypassed by anyone who sends HTTP requests directly. For this reason, authoritative validation must happen on the server.

B3. Proposed services

Architecture

The proposed solution has **one backend application** with a REST API. Both clients talk only to this API. The backend is organized in layers, but it is a **single deployable service**. The logical layers are *not* separate microservices.

Component	Responsibility
Web application	Presentation and user experience for browsers
Mobile application	Presentation and user experience for phones
Shared REST API layer (controllers)	Routing, authentication, authorization checks, request parsing, schema validation, converting results to JSON, choosing HTTP status codes
Business logic layer	The rules: eligibility, late fee calculation, duplicate check, seat availability, state changes
Data access layer	Reading and writing data through repositories or an ORM, using parameterized queries and transactions
Database	Persistent storage and last-line integrity constraints (for example, a unique constraint on student and workshop)

How clients communicate with the API

Both clients use HTTPS requests with JSON bodies and receive JSON responses. They send an access token with each request. The API contract (OpenAPI) is the same for both clients. For example:

Operation	Request
List workshops	`GET /api/v1/workshops?available=true`
Get one workshop	`GET /api/v1/workshops/{id}`
Register	`POST /api/v1/registrations`
List my registrations	`GET /api/v1/registrations`
Cancel a registration	`DELETE /api/v1/registrations/{id}`

How business logic is centralized

The rules are implemented **once**, in the business logic layer. The API controllers do not contain rules. They pass the request to the business layer and translate the result into HTTP. Clients never calculate fees or decide eligibility. They send the user's intention ("register me") and show the result that the server returns.

How the backend accesses persistent data

The business logic layer asks the data access layer for data. The data access layer is the only code that talks to the database. It uses a limited database account, parameterized queries (to prevent SQL injection), and transactions. A seat check and a registration insert happen in the **same transaction**, which prevents overbooking.

Responsibilities: client vs. server

Responsibility	Client (web or mobile)	Server (API and backend)
Screens, navigation, layout	Yes	No
Quick input hints (for example, "email looks incomplete")	Yes, for faster feedback only	No
Authoritative validation	No	**Yes**
Calculations (fees, totals)	No, only display the server's result	**Yes**
Permission decisions	No	**Yes**
Seat and duplicate checks	No	**Yes**, inside a transaction
Data persistence	No	Yes
Handling loading, errors, and offline state	Yes	Provides clear status codes and messages
Secure token storage	Yes (secure storage on the device)	Issues and verifies tokens

A client may keep small convenience checks (such as "this field is required") to give instant feedback. These checks are never trusted. The server repeats and enforces every rule.

B4. Architecture diagram

Solid arrows show requests and calls. Dashed arrows show results returned inside the backend. A double-headed arrow between a client and the API means that the client sends an HTTPS request and receives a JSON response with an HTTP status code on the same connection.

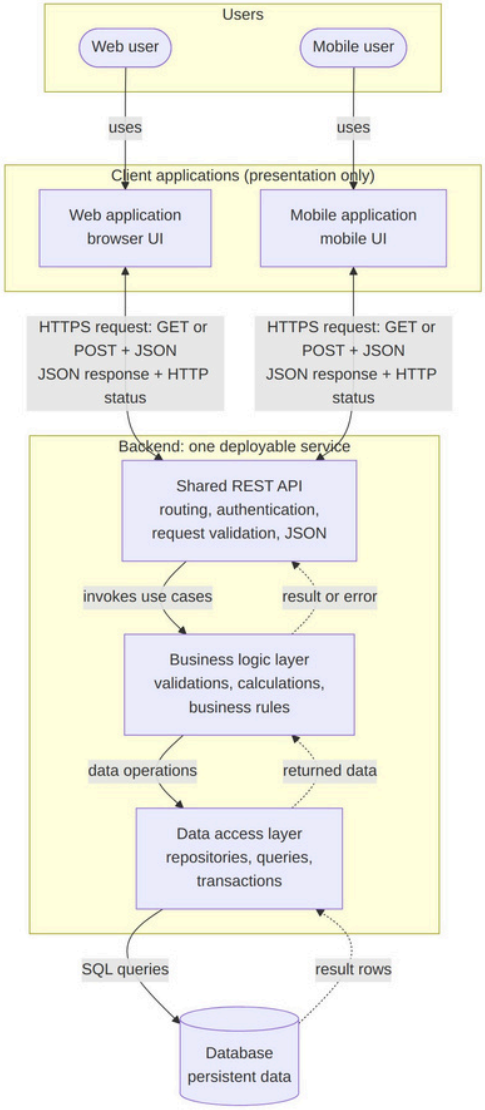


Figure 1. Architecture of the shared REST API solution (Case 4)

How to read the diagram. A web or mobile user interacts with a client application. The client sends an HTTPS request to the shared REST API. The API calls the business logic layer, which applies the rules and uses the data access layer to read or write the database. The result travels back through the same layers

and the API returns a JSON response with an HTTP status code. The web client and the mobile client use the same entry point and therefore the same rules.

B5. Data exchanged

GET request: list available workshops

```
GET /api/v1/workshops?available=true&page=1&pageSize=10 HTTP/1.1
Host: api.campus-workshops.example.edu
Accept: application/json
Authorization: Bearer <access-token>
```

Successful response to the GET request

```
HTTP/1.1 200 OK
Content-Type: application/json
```


```
{
  "items": [
    {
      "id": "ws_101",
      "title": "Introduction to Git",
      "startsAt": "2026-11-12T16:00:00Z",
      "seatsAvailable": 8,
      "price": { "amount": "0.00", "currency": "MXN" }
    },
    {
      "id": "ws_204",
      "title": "Docker Basics",
      "startsAt": "2026-11-14T16:00:00Z",
      "seatsAvailable": 1,
      "price": { "amount": "150.00", "currency": "MXN" }
    }
  ],
  "page": 1,
  "pageSize": 10,
  "totalItems": 2 }
```


Money amounts are strings, which avoids floating-point rounding errors in JSON numbers.

POST request: register for a workshop

```
POST /api/v1/registrations HTTP/1.1
Host: api.campus-workshops.example.edu
Content-Type: application/json
Accept: application/json
Authorization: Bearer <access-token>
```

```
{
  "workshopId": "ws_204",
 "contactPhone": "+526145550123"
}
```

The client does **not** send the fee. The server calculates it.

Successful response to the POST request

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/registrations/reg_5521
```

```
{
  "id": "reg_5521",
 "workshopId": "ws_204",
"status": "CONFIRMED",
"fee": {
      "baseAmount": "150.00",
      "lateFee": "15.00",
      "total": "165.00",
      "currency": "MXN"
 },
 "createdAt": "2026-11-13T15:04:05Z" }
```

Web and mobile clients receive the same total (165.00) because the server is the only place where the late fee is calculated.

Error response: invalid content

This example uses the Problem Details format for HTTP APIs (Nottingham et al., 2023), which gives clients a predictable error structure.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
```

```
{
  "type": "https://api.campus-workshops.example.edu/problems/validation-error",
 "title": "The request content is not valid",
"status": 422,
 "detail": "One or more fields are invalid.",
"instance": "/api/v1/registrations",
"errors": [
    {
      "field": "contactPhone",
      "message": "Use international format, for example +526145550123."
    }
  ]
}
```


Another useful error is a business-rule conflict, such as a full workshop:


```
{
  "type": "https://api.campus-workshops.example.edu/problems/workshop-full",
 "title": "The workshop is full",
 "status": 409,
 "detail": "No seats are available for workshop ws_204.",
"instance": "/api/v1/registrations"
}
```


Notice that neither error message exposes internal details such as stack traces, SQL, or table names.

Public, personal, and sensitive information

Category	Examples in this system	Handling
Public	Workshop title, date, description, seats available, public price	Can be returned to any authenticated client (or even without login, if the team decides so)
Personal	Full name, email, phone number, student ID, a student's registrations	Returned only to the owner or to authorized staff. Collect only what is needed. Do not write it into logs.
Sensitive	Passwords and password hashes, access and refresh tokens, API keys, payment details, internal notes, security answers	Never returned by the API. Stored hashed or encrypted. Never logged.

Information that must not be exposed to clients

- Password hashes, salts, tokens, and secrets.
- Other users' personal data (for example, a list of everyone registered in a workshop, shown to a regular student).
- Internal database fields, such as internal IDs, `is_admin`, or audit columns, unless the client truly needs them.
- Internal error details: stack traces, SQL statements, server paths, and library versions.
- Internal business configuration that attackers could abuse, such as exact rate-limit thresholds.

The API should return **purpose-built response objects** (sometimes called DTOs) with an explicit list of allowed fields. It should never serialize the full database record by default. This also helps to avoid the "Broken Object Property Level Authorization" risk described in B7.

B6. Failure modes

Centralizing the logic creates a new dependency: both clients now need the API. For this reason, the design must plan for failure. The table below analyzes five failure modes.

Failure mode	Impact	Mitigation
REST API unavailable (service crashed, deployment error, overload)	Both clients lose all write operations and any data they do not already have. All users are affected at the same time.	Run at least two instances behind a load balancer. Add a health-check endpoint and automatic restart. Return `503 Service Unavailable` with `Retry-After` when overloaded. Clients show a clear message ("The service is temporarily unavailable. Please try again in a few minutes.") and may show recently loaded data marked as "last updated at...". Alert the team from monitoring.
Network connection failure (no signal, Wi-Fi lost, DNS error)	The request never reaches the API, or the response never reaches the client. The client does not know whether the operation happened.	Detect the offline state and tell the user. Retry **safe reads** a limited number of times. For writes, do not retry blindly: use an idempotency key (see below) or let the user confirm before sending again. After reconnecting, the client checks the current state (for example, `GET /api/v1/registrations`) before repeating a write.
Database unavailable (connection pool exhausted, DB down, failover)	The API is running but cannot read or write data. Requests fail or become slow.	Use connection timeouts and a pool limit so requests fail fast instead of piling up. Return `503` (not `500` with a stack trace) for known infrastructure failures. Use a circuit breaker so the API stops hammering a broken database (Microsoft, n.d.-a). Keep regular backups and a tested recovery plan. Log the error with a request ID and alert the team.
Request timeout (slow query, overloaded server, slow network)	The user waits and may press the button again. A write may have been completed on the server even though the client received no answer.	Set explicit client timeouts (for example 10 seconds for reads and 15 seconds for writes, tuned with real measurements) and server-side timeouts. Show a loading indicator and disable the submit button while a request is in progress. For writes, use an idempotency key so a repeated request does not create a duplicate.
Unexpected server response (HTML error page from a proxy, invalid JSON, missing field, unknown status code)	The client may crash or display wrong data.	The client must check the status code and the `Content-Type` before parsing, handle parsing errors, and tolerate unknown extra fields. Show a generic friendly error and log technical details. Use contract tests against the OpenAPI document so changes do not silently break clients. The server returns errors in a consistent format (Nottingham et al., 2023).

Retry rules

Retries can fix temporary problems, but they can also cause damage. Microsoft's guidance for the Retry pattern is to retry only transient faults, to wait between attempts, to stop after a limited number of attempts, to log failures, and to consider whether the operation is idempotent. If it is not, a retry might execute it more than once (Microsoft, n.d.-c).

Request type	Automatic retry?	Policy
`GET` (read, safe and idempotent)	Yes, limited	Up to 2 retries (3 attempts in total) for network errors, `502`, `503`, `504`, and `429`. Use exponential backoff with random jitter (for example about 1 s, then 2 s). Respect `Retry-After` when present.
`PUT` or `DELETE` (idempotent by definition)	Yes, limited	Same policy as `GET`, because repeating the request has the same intended effect (MDN Contributors, n.d.-b).
`POST` that creates something (for example, a registration or payment)	**No automatic retry by default**	A retry could create a duplicate. Retry only when the API supports an **idempotency key**. Otherwise show the error and let the user decide, after checking the current state.
Any request after `400`, `401` `403` `404` `409` `422`, or	No	These are not transient. Retrying will give the same answer. Fix the request or ask the user to act.

Never retry without limit. Unlimited retries from many clients can turn a short outage into a bigger one.

Idempotency key for safe `POST` retries

The client generates a unique value (for example, a UUID) for each user action and sends it in a header, such as `Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7`. The server stores the key together with the result. If the same key arrives again, the server returns the stored result instead of creating a second registration. This is a common practice among APIs that handle payments and orders, and it is a design option the team can add to `POST /api/v1/registrations`. A database unique constraint (student and workshop) is a second safety net.

Understandable error messages

Technical situation	Message to the user
No network	"You appear to be offline. Check your connection and try again."
`503` or API unreachable	"The service is temporarily unavailable. Please try again in a few minutes."
Timeout on a write	"We could not confirm your registration. Please check 'My registrations' before trying again."
`409` workshop full	"This workshop has no seats left."
Unexpected response	"Something went wrong. Please try again. If it continues, contact support with code 8F3A."

The code in the last row is the request ID (correlation ID), which lets support find the matching server log entry.

Logging

- The server logs every failed request with a **request ID**, timestamp, route, status code, and duration.
- Logs never include passwords, tokens, or full personal data.
- Clients can send crash and error reports that include the request ID.
- Alerts are created for high error rates and for repeated `5xx` responses.

B7. Security

The proposed architecture has one important property: **all rules and data pass through the API**. Therefore, the API is the main security boundary and must be protected carefully.

Required mechanisms

Mechanism	What it means in this architecture
HTTPS	All traffic between clients and the API uses TLS. Plain HTTP is blocked or redirected. Certificates are renewed automatically. Mobile apps should not accept invalid certificates.
Authentication	Users sign in, and the API issues short-lived access tokens (for example, JWT access tokens) and, if needed, refresh tokens, following OAuth 2.0 patterns (Hardt, 2012; Jones et al., 2015). Web and mobile use the same mechanism. Mobile apps store tokens in the platform's secure storage. Public clients use Authorization Code with PKCE (Sakimura et al., 2015). Passwords are stored only as salted hashes created with a slow algorithm such as Argon2id or bcrypt (OWASP Foundation, n.d.-b).
Authorization	Every request is checked on the server for **who** is calling and **what** they may access. Two checks are needed: function-level ("may this role call this endpoint?") and object-level ("does this registration belong to this user?"). The client's role or UI state is never trusted.
Input validation	The API validates every input against a schema: type, length, range, format, and allowed values. It uses allow-lists and rejects unknown or malformed fields. All SQL uses parameterized queries. Validation is repeated on the server even if the client already checked (OWASP Foundation, n.d.-a).
Data protection	Collect only the data that is needed. Encrypt sensitive data at rest where possible. Do not log secrets. Return only the fields the client needs. Separate environments (development and production) and keep secrets out of the repository.
Rate limiting	Apply limits per user, per token, and per IP address, with stricter limits for sensitive routes such as login and registration. Return 429 with Retry-After (Nottingham & Fielding, 2012).
Secure error handling	Return generic messages and a request ID. Use the Problem Details format (Nottingham et al., 2023). Never return stack traces, SQL errors, or internal paths. Use `401` and `403` correctly and avoid messages that reveal whether an account exists.

Mechanism	What it means in this architecture
Database access control	Only the backend connects to the database. The database user has **least privilege** (no schema-changing rights in production). The database is not exposed to the internet and accepts connections only from the backend network. Backups are encrypted.

For general practical guidance on securing REST services, see the OWASP REST Security Cheat Sheet (OWASP Foundation, n.d.-c).

Relevant risks from the OWASP API Security Top 10 (2023)

The OWASP API Security Top 10 is a list of the most critical API security risks (OWASP Foundation, 2023). The table below keeps only the risks that clearly apply to this architecture.

OWASP risk	Why it applies here	Mitigation
API1:2023 Broken Object Level Authorization	The API exposes IDs such as `/registrations/reg_5521`. If the server does not check ownership, a student could change the ID and read or cancel another student's registration.	Check on every object access that the authenticated user owns the object or has a role that allows access. Use unpredictable IDs as an extra layer, not as the only protection. Add automated tests for cross-user access.
API2:2023 Broken Authentication	Both clients depend on tokens. Weak login, long-lived tokens, or poor token validation would let attackers impersonate users on both clients at once.	Use a proven authentication library or provider. Verify signature, algorithm, and expiry of every token. Use short access-token lifetimes. Limit login attempts. Store tokens securely on the device.
API3:2023 Broken Object Property Level Authorization	Returning full database objects can expose properties such as `isAdmin` or internal notes (excessive data exposure). Accepting a full object in `POST` or `PUT` can let a user change properties they should not control (mass assignment).	Use explicit response objects that list the allowed fields. Accept only the expected input fields and ignore or reject all others. Never bind request JSON directly to a database model.
API4:2023 Unrestricted Resource Consumption	A public API can be flooded with requests, very large payloads, or expensive queries (for example, listing without pagination). This can cause downtime and extra cost.	Use rate limits, maximum body sizes, request timeouts, required pagination with a maximum `pageSize`, and monitoring of resource use.
API5:2023 Broken Function Level Authorization	The API may have both normal and administrative functions, for example, "list all registrations" or "cancel any registration". If the server does not check roles, normal users can call them.	Deny by default. Check the role for every function. Keep administrative routes separate and test them with non-admin accounts.
API6:2023 Unrestricted Access to Sensitive Business Flows	Registration for a limited number of seats is a business flow that automated scripts can abuse, for example by reserving all seats.	Apply stricter rate limits to the registration flow, require authentication, detect unusual patterns, and use limits per user.

OWASP risk	Why it applies here	Mitigation
API8:2023 Security Misconfiguration	A shared API for several clients has many settings: CORS, HTTP headers, debug mode, TLS, default accounts, and error output. A mistake affects every client.	Use hardened, repeatable configuration. Disable debug output and default credentials in production. Restrict CORS to known origins. Review configuration in code review.
API9:2023 Improper Inventory Management	Several client versions will call the API for a long time. Old or undocumented API versions, test endpoints, and forgotten routes can stay online and unprotected.	Keep an up-to-date OpenAPI document, version the API (/api/v1), define a deprecation plan, and remove unused versions and routes.

The remaining OWASP risks are not included, because they do not clearly apply to this case. For example, API7 (Server-Side Request Forgery) and API10 (Unsafe Consumption of APIs) are mainly relevant when the backend fetches remote resources or calls third-party APIs, which Case 4 does not require.

B8. Trade-offs

A shared API solves the duplication problem, but it also has costs. The table below explains each one and a practical response.

Trade-off	Explanation	Response
Central API dependency	Both clients now need the API to work. If clients contain no business rules, they cannot perform most actions alone.	Make the API reliable (see B6). Allow read-only access to recently cached data for non-critical screens.
Potential single point of failure	One API failure affects web and mobile users at the same time.	Run multiple instances, use health checks and automatic restarts, keep a deployment rollback plan, and monitor availability.
Network latency	Operations that were local in the client now require a network round trip. Slow mobile networks make this more visible.	Keep responses small, use pagination, compress responses, and use HTTP caching for data that rarely changes. Show loading states. Avoid calling the API once per field.
Infrastructure costs	A server, a database, TLS certificates, backups, and monitoring have a cost, and the API must be available at all times.	Start with one small backend and scale only when measurements justify it. A single backend is cheaper than a set of microservices.
Backend maintenance	One backend becomes critical. Poor structure or missing tests will hurt both clients.	Use clear layers, automated tests for business rules, code review, and documentation. The code base is still smaller than two duplicated implementations.
Monitoring	Problems now appear on the server, and clients are only the place where users notice them.	Use centralized logs with request IDs, metrics (error rate, latency, traffic), and alerts.

Trade-off	Explanation	Response
API versioning	Changing a response or rule can break clients that were not updated.	Use a version prefix (`/api/v1`). Prefer additive, backward-compatible changes (new optional fields). Introduce `/api/v2` only for breaking changes. Announce deprecations early.
Compatibility between clients and API versions	Web users get the newest version immediately, but mobile users may use old app versions for months.	Support the current and previous API versions at the same time. Let the API report a minimum supported app version so very old apps can ask users to update. Use contract tests in the pipeline.

Evaluation: is the proposed solution appropriate?

Yes, the solution is appropriate for Case 4. The root problem is the duplication of business rules. Moving the rules into one REST API gives a single source of truth, and any change to a rule is made once and becomes visible to both clients. Consistency of validation, calculation, and authorization improves, and the database receives data that was checked in one place. Security is also easier to manage because one boundary can be hardened and monitored.

The costs are real but manageable. They mainly concern availability, latency, and versioning discipline, and each has a standard mitigation. These costs are lower than the cost of keeping two diverging implementations.

The solution is also **proportionate**. It uses a single backend with internal layers, not a large set of microservices. For a small or medium system with one team, this keeps the architecture simple. If the system later grows, for example if one function needs to scale separately or a separate team owns it, the backend can be split at that point. Microservices should be a response to a measured need, not the starting assumption.

Part C — Application to our project (DevPath)

This part refers to **our own university project, DevPath**, and not to the illustrative scenario of Case 4. It uses the same ideas as Case 4 (a clear contract, one place for the logic, and a well-defined HTTP interface), but it solves a different problem: isolating heavy text analysis from the main API.

Source of the project information: the DevPath component map (DevPath team, 2026) and the team's design decisions confirmed for this activity.

C1. Project and module supported by the microservice

DevPath helps recent software graduates and junior developers compare their skills with job descriptions and build a learning path. The technology stack is React + Vite (frontend), Node.js + Express (backend), PostgreSQL (database), and Git + GitHub (repository), with the interface designed in Figma (DevPath team, 2026).

Item	Detail
Project module supported	**Job Compatibility Analyzer**, the module behind the *Job Analysis* screen
Own microservice (Evidence 3)	**Skill Matching Microservice** (named *Text Analysis & Skill Matching Engine* in the component map)
What it does	Extracts technologies from job text, compares them with the user's skills, and returns matches, gaps, and a compatibility percentage
Why a separate microservice	Text analysis is heavy work. Isolating it keeps the main API responsive and lets this service scale separately (DevPath team, 2026).
Third-party service in the project	JSearch API (via RapidAPI), used by the Express backend to retrieve real junior-level job listings

C2. Architecture decisions confirmed by the team

Topic	Decision
State	The microservice is **stateless**. It receives data, processes it, and returns JSON. It stores nothing.
Database access	The microservice has **no direct access to PostgreSQL**.
Who calls it	**Only the Express backend** calls the microservice. The React frontend never calls it directly.
User skills	Express reads the user's skills from PostgreSQL and sends them in the request body.

Topic	Decision
Technology catalog	The catalog is stored in PostgreSQL and managed by Express. It standardizes names such as `JS`, `Javascript`, and `JavaScript` as one technology. Express sends the catalog data to the microservice when needed. Catalog management endpoints belong to the main backend, not to the microservice.
Source of `jobText`	Two sources: (1) the user pastes a job description, or (2) Express retrieves a vacancy description from JSearch. In both cases, Express sends the text in the `jobText` field.
Analysis history	If the history is needed, Express saves it in PostgreSQL.

C3. Request flow

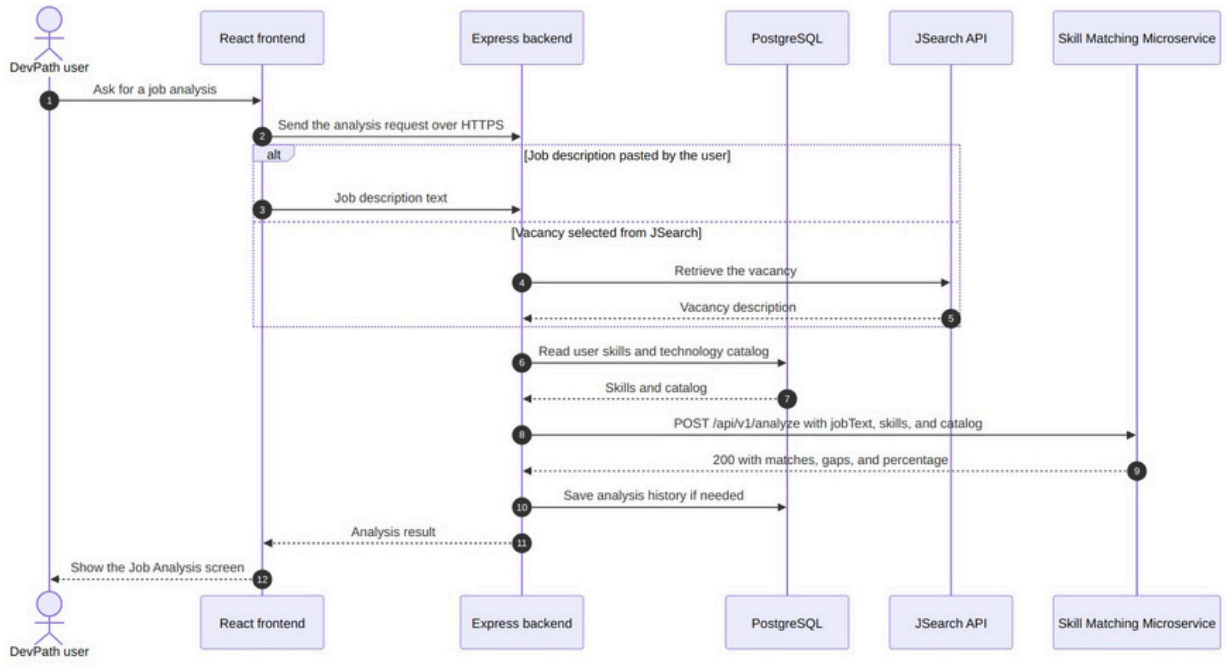


Figure 2. Request flow of the DevPath Skill Matching Microservice

C4. Conventions for the microservice

Convention	Value
Base path and version	`/api/v1/`
Data format	JSON (`Content-Type: application/json`) for requests and responses
Methods	`GET` and `POST`, depending on the operation. The three initial endpoints use `POST`.
Success code	`200 OK`
Invalid request	`400 Bad Request`

Convention	Value
Internal error	`500 Internal Server Error`
Additional codes (proposed)	`401` when the caller's service key is missing or wrong, and `413` when the body is too large
Error body	Problem Details JSON, `application/problem+json` (Nottingham et al., 2023)
Caller authentication (proposed)	Express sends a shared service key in the `X-API-Key` header. The key is stored in an environment variable, and the service is reachable only from the backend network.

These are the team's initial conventions, and they apply unless the teacher indicates otherwise.

C5. The three initial endpoints

Method	Endpoint	Input (body or query)	Output (success)	Errors
POST	`/api/v1/analyze`	Body: `{ jobText, skills:[{ catalogId, name}], technologyCatalog: [ { catalogId, name, aliases}] }`	`200 { technologiesFound: [ { catalogId, name, mentions } ], matches: [ { catalogId, name } ], gaps: [ { catalogId, name } ], compatibilityPercentage }`	400, 401, 413, 500
POST	`/api/v1/match`	Body: `{ skills: [ { catalogId, name } ], requirements:[{ catalogId, name } ] }`	`200 { matches: [ { catalogId, name } ], gaps: [ { catalogId, name } ], compatibilityPercentage }`	400, 401, 500
POST	`/api/v1/extractions`	Body: `{ jobText, technologyCatalog: [ { catalogId, name,aliases}] }`	`200 { technologies: [ { catalogId, name, mentions } ] }`	400, 401, 413, 500

Endpoint	Single job
`/api/v1/analyze`	Analyzes a job description and calculates the user's compatibility with it in one call. This is the main endpoint used by the Job Analysis screen.
`/api/v1/match`	Compares a list of user skills with a list of technology requirements. Useful when the requirements are already known.
`/api/v1/extractions`	Extracts only the technologies found in a job description.

`/analyze` combines the logic of `/extractions` and `/match`. Internally, the service can use the same two functions for all three endpoints.

C6. Endpoint specifications

Shared data objects

Object	Fields	Meaning
Catalog item	`catalogId` (integer), `name` (string), `aliases` (array of strings)	One standardized technology and the other names used for it
Skill or requirement	`catalogId` (integer), `name` (string)	A technology in the catalog. Comparison uses `catalogId`.

Compatibility percentage: `compatibilityPercentage = (number of matched requirements ÷ number of requirements) × 100`, rounded to one decimal place.

Suggested extraction approach (deterministic, no machine learning): convert the text to lowercase, search for each catalog name and alias as a whole term (so `Java` is not found inside `JavaScript`), try longer terms first, count the mentions, and return each technology once. Terms with symbols, such as `C++`, `C#`, and `Node.js`, need careful matching.

Proposed input limits (to be confirmed): `jobText` between 1 and 20,000 characters, `technologyCatalog` with 1 to 1,000 items, and `skills` with 0 to 500 items.

1. `POST /api/v1/analyze`

Purpose: extract technologies from a job description, compare them with the user's skills, and return matches, gaps, and a compatibility percentage.

Request:

```
POST /api/v1/analyze HTTP/1.1
Content-Type: application/json
X-API-Key: <service-key>
```

```
{
  "jobText": "Junior Frontend Developer. We need experience with React, JavaScript (JS),
Git, and basic Docker knowledge. You will also consume REST APIs.",
  "skills": [
    { "catalogId": 2, "name": "React" },
    { "catalogId": 3, "name": "Git" }
  ],
  "technologyCatalog": [
    { "catalogId": 1, "name": "JavaScript", "aliases": ["JS", "Javascript"] },
    { "catalogId": 2, "name": "React", "aliases": ["React.js", "ReactJS"] },
    { "catalogId": 3, "name": "Git", "aliases": [] },
    { "catalogId": 4, "name": "Docker", "aliases": [] },
    { "catalogId": 5, "name": "REST API", "aliases": ["REST", "RESTful", "REST APIs"] }
  ]
}
```

Successful response (`200 OK`). Technologies are listed in order of first appearance in the text:


```
{
  "technologiesFound": [
    { "catalogId": 2, "name": "React", "mentions": 1 },
    { "catalogId": 1, "name": "JavaScript", "mentions": 2 },
    { "catalogId": 3, "name": "Git", "mentions": 1 },
    { "catalogId": 4, "name": "Docker", "mentions": 1 },
    { "catalogId": 5, "name": "REST API", "mentions": 1 }
  ],
  "matches": [
    { "catalogId": 2, "name": "React" },
    { "catalogId": 3, "name": "Git" }
  ],
  "gaps": [
    { "catalogId": 1, "name": "JavaScript" },
    { "catalogId": 4, "name": "Docker" },
    { "catalogId": 5, "name": "REST API" }
  ],
  "compatibilityPercentage": 40.0
}
```


Calculation: 2 matched technologies out of 5 required technologies is 40.0%. If the text contains no known technology, the service still answers `200` with empty lists and `"compatibilityPercentage": null`, because there is nothing to compare.

Error	When it happens
400	InvalidJSON, `jobText` missing or empty, `skills` or `technologyCatalog` missing or with the wrong type, empty `technologyCatalog`
401	Service key missing or incorrect
413	Request body or `jobText` larger than the allowed limit
500	Unexpected internal error

2. `POST /api/v1/match`

Purpose: compare the user's skills with a list of required technologies.

Request:

```
{
  "skills": [
    { "catalogId": 2, "name": "React" },
    { "catalogId": 3, "name": "Git" }
 ],
"requirements": [
    { "catalogId": 1, "name": "JavaScript" },
    { "catalogId": 2, "name": "React" },
    { "catalogId": 3, "name": "Git" },
    { "catalogId": 4, "name": "Docker" }
  ]
}
```

Successful response (`200 OK`). The results follow the order of `requirements`:

```
{
  "matches": [
    { "catalogId": 2, "name": "React" },
    { "catalogId": 3, "name": "Git" }
 ],
"gaps": [
    { "catalogId": 1, "name": "JavaScript" },
    { "catalogId": 4, "name": "Docker" }
 ],
 "compatibilityPercentage": 50.0
}
```

Error	When it happens
400	Invalid JSON, `skills` or `requirements` missing or with the wrong type, `requirements` empty, an item without `catalogId`
401	Service key missing or incorrect
500	Unexpected internal error

An empty `skills` list is valid. In that case all requirements become gaps and the percentage is `0.0`.

3. POST /api/v1/extractions

Purpose: find the technologies mentioned in a job description.

Request:

```
{
  "jobText": "We are hiring a junior developer with Javascript and React.js experience. Knowledge of Docker is a plus.",
  "technologyCatalog": [
    { "catalogId": 1, "name": "JavaScript", "aliases": ["JS", "Javascript"] },
    { "catalogId": 2, "name": "React", "aliases": ["React.js", "ReactJS"] },
    { "catalogId": 4, "name": "Docker", "aliases": [] }
  ]
}
```

Successful response (`200 OK`):

```
{
  "technologies": [
    { "catalogId": 1, "name": "JavaScript", "mentions": 1 }, {
    "catalogId": 2, "name": "React", "mentions": 1 },
    { "catalogId": 4, "name": "Docker", "mentions": 1 }
  ]
}
```

Error	When it happens
400	Invalid JSON, `jobText` missing or empty, `technologyCatalog` missing or empty
401	Service key missing or incorrect
413	`jobText` or request body larger than the allowed limit
500	Unexpected internal error

Error response example (`400 Bad Request`)

```
{
  "type": "https://devpath.example/problems/invalid-request",
  "title": "The request is not valid",
  "status": 400,
  "detail": "The field jobText must be a non-empty string.",
  "instance": "/api/v1/analyze"
}
```

C7. Design notes and points for teacher approval

Why POST **for read-only analysis?** The input is a large block of text that may contain job-related content, which does not belong in a URL (URLs have length limits and can be recorded in logs). `POST` allows a request body. These requests have no side effect on stored data, because the service is stateless. Express can therefore retry them safely, with a small retry limit and a timeout (see B6).

Naming of the endpoints. `/extractions` is a noun. `/analyze` and `/match` are verbs. The course guide says that in REST the verb should come from HTTP, not from the URL (UTCH BIS, 2026). The team chose these names because both endpoints perform a stateless calculation and do not create or store a resource.

If the teacher prefers names that are strictly noun-based, the direct alternatives are `POST /api/v1/analyses` and `POST /api/v1/matches`. The behavior and the data would stay the same.

Items proposed by us that need confirmation before Evidence 3:

1. The use of a shared service key (`X-API-Key`) between Express and the microservice, and the resulting `401` response.
2. The input limits (for example, 20,000 characters for `jobText`) and the `413` response.
3. Sending the technology catalog in each request, instead of loading it once by another method.
4. The endpoint names discussed above.

Implementation scope. The endpoints are specified here as a design. They have not been implemented. Implementation is planned for Evidence 3, after the teacher approves this specification. A natural next step is to write the same contract as an OpenAPI 3.1 file (`openapi.yaml`), so that the Express backend team and the microservice team can work in parallel (OpenAPI Initiative, 2021).

References

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8259

Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., & Yergeau, F. (Eds.). (2008). *Extensible markup language (XML) 1.0* (5th ed.). World Wide Web Consortium. https://www.w3.org/TR/xml/

Chinnici, R., Moreau, J.-J., Ryman, A., & Weerawarana, S. (Eds.). (2007). *Web services description language (WSDL) version 2.0 part 1: Core language*. World Wide Web Consortium. https://www.w3.org/TR/wsdl20/

DevPath team. (2026). *DevPath component map* [Unpublished project document]. Desarrollo Web Integral (IDGSBIS91N), SD2026.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

GraphQL Foundation. (n.d.). *Introduction to GraphQL*. Retrieved October 30, 2026, from https://graphql.org/learn/

Gudgin, M., Hadley, M., Mendelsohn, N., Moreau, J.-J., Nielsen, H. F., Karmarkar, A., & Lafon, Y. (Eds.). (2007). *SOAP version 1.2 part 1: Messaging framework* (2nd ed.). World Wide Web Consortium. https://www.w3.org/TR/soap12-part1/

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6749

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON web token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

MDN Contributors. (n.d.-a). *429 Too Many Requests*. MDN Web Docs. Retrieved October 30, 2026, from https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/429

MDN Contributors. (n.d.-b). *Idempotent*. MDN Web Docs. Retrieved October 30, 2026, from https://developer.mozilla.org/en-US/docs/Glossary/Idempotent

Microsoft. (n.d.-a). *Circuit breaker pattern*. Microsoft Learn. Retrieved October 30, 2026, from https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

Microsoft. (n.d.-b). *Microservices architecture style*. Microsoft Learn. Retrieved October 30, 2026, from https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices

Microsoft. (n.d.-c). *Retry pattern*. Microsoft Learn. Retrieved October 30, 2026, from https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

Nottingham, M., & Fielding, R. (2012). *Additional HTTP status codes* (RFC 6585). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6585

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9457

OpenAPI Initiative. (2021). *OpenAPI specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0.html

OWASP Foundation. (2023). *OWASP API security top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

OWASP Foundation. (n.d.-a). *Input validation cheat sheet*. OWASP Cheat Sheet Series. Retrieved October 30, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *Password storage cheat sheet*. OWASP Cheat Sheet Series. Retrieved October 30, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). *REST security cheat sheet*. OWASP Cheat Sheet Series. Retrieved October 30, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

Rescorla, E. (2018). *The transport layer security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8446

Sakimura, N., Bradley, J., & Agarwal, N. (2015). *Proof key for code exchange by OAuth public clients* (RFC 7636). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7636

Sheffer, Y., Hardt, D., & Jones, M. (2020). *JSON web token best current practices* (RFC 8725). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8725

UTCH BIS. (2026). *Unit 3 · Activity 3: Case study — Web services and microservices in action* [Course activity guide]. Desarrollo Web Integral (IDGSBIS91N), SD2026.